



# Модуль `testing_lab`

---

## Назначение

---

Модуль рассчитывает сумму по выбранным позициям заказа и создаёт заявку на возврат с проверкой статуса заказа и срока возврата. Заявка сохраняется в SQLite через `ReturnRepository`.

## Основной сценарий

---

1. Передать позиции заказа и выбранные количества в `calculate_refund`.
2. `RefundService.create_request` проверит, что заказ получен и срок возврата не истёк, затем рассчитает сумму.
3. Сервис сохранит заявку через репозиторий. Новая заявка получает статус `requested`.

## Роли

---

- `calculate_refund(items: list[dict], requested: dict[int, int]) -> Decimal` — проверяет выбранные позиции и количества, вычисляет сумму с учётом скидки и округляет её до двух знаков.
- `ReturnRepository(connection: sqlite3.Connection)` — создаёт таблицу `returns`, сохраняет заявки и возвращает заявку по идентификатору.
- `RefundService(repository: ReturnRepository)` — проверяет условия возврата, вызывает расчёт суммы и передаёт заявку в репозиторий. Метод `create_request(order: dict, requested: dict[int, int], today: date) -> dict` создаёт заявку.

## Ключевые бизнес-ограничения

---

- Нужно выбрать хотя бы одну позицию; товар должен присутствовать в заказе.
  - Возвращаемое количество должно быть больше нуля и не превышать купленное. Вернуть всё купленное количество разрешено.
  - Сумма рассчитывается по цене позиции из заказа; при сохранённой скидке учитывается фактически оплаченная цена. Результат имеет точность до двух знаков после запятой.
  - Возврат доступен только для заказа со статусом `delivered` и известной датой `delivered_at`.
  - Дата получения не может быть в будущем относительно переданной даты `today`. Возврат разрешён по 14-й календарный день включительно.
  - Успешно созданная заявка сохраняется с суммой расчёта и начальным статусом `requested`.
-

## Структура файлов

---

```
testing_lab/
├── __init__.py
├── API.md
├── README.md
├── CONTRACT.md
└── refunds.py

tests/testing_lab/
├── conftest.py
├── test_refunds_unit.py
└── test_refunds_integration.py
```

## Минимальный пример

---

```
import sqlite3
from datetime import date
from decimal import Decimal

from testing_lab.refunds import RefundService, ReturnRepository, calculate_refund

items = [{"product_id": 101, "quantity": 2, "unit_price": Decimal("25.00")}]
requested = {101: 1}

amount = calculate_refund(items, requested)
order = {
    "id": 1001,
    "status": "delivered",
    "delivered_at": date(2026, 9, 1),
    "items": items,
}

connection = sqlite3.connect(":memory:")
repository = ReturnRepository(connection)
service = RefundService(repository)
created = service.create_request(order, requested, date(2026, 9, 2))

assert amount == Decimal("25.00")
assert created["amount"] == amount
connection.close()
```

## Запуск тестов

---

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab -v
```

## Branch coverage

---

```
.\.venv\Scripts\python.exe -m coverage erase
.\.venv\Scripts\python.exe -m coverage run --branch --source=testing_lab -m pytest tests\
testing_lab
.\.venv\Scripts\python.exe -m coverage report -m
```

[Справочник API](#) · [Контракт модуля](#)